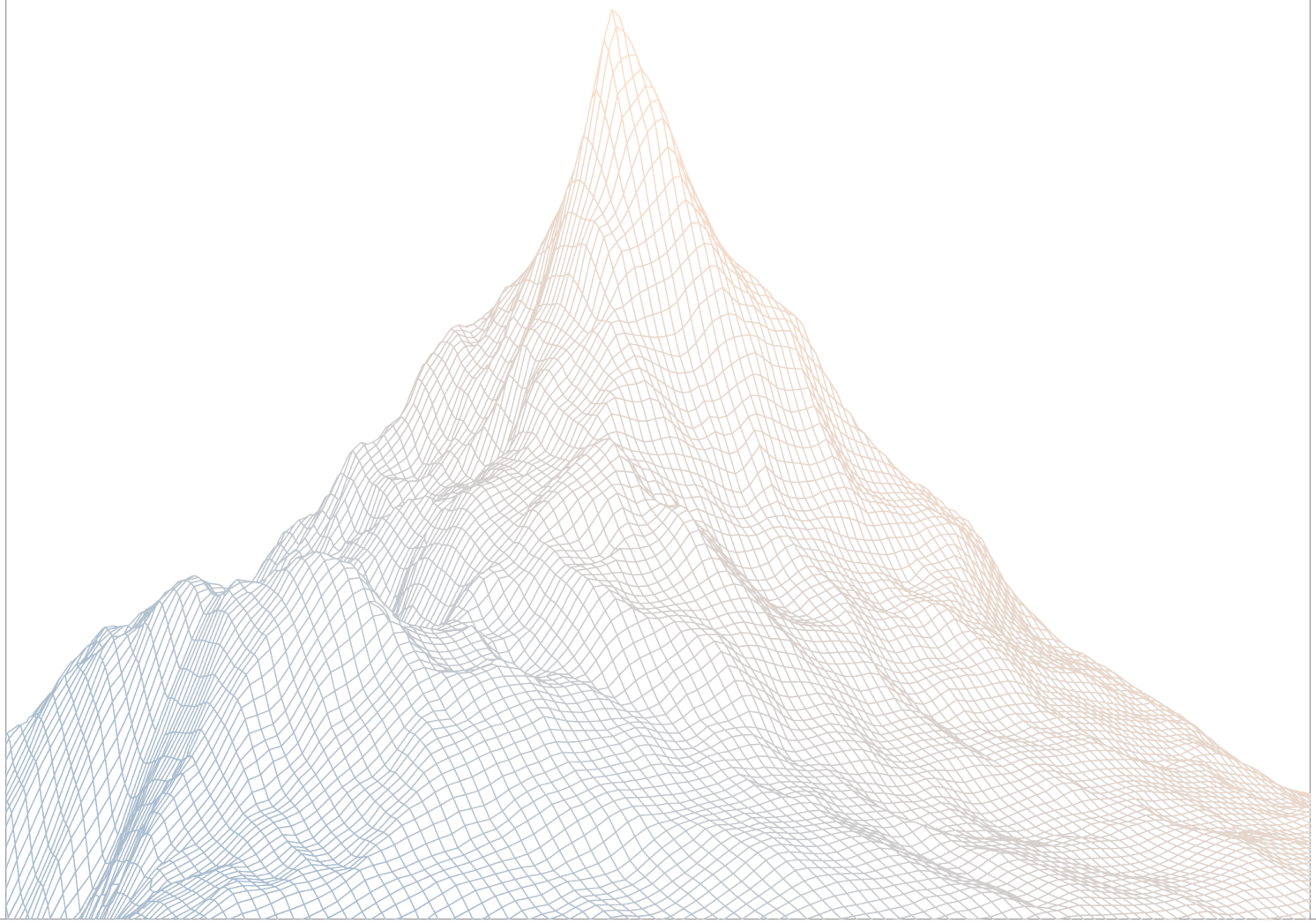
Zenith

Meteora

Smart Contract Security Assessment

VERSION 1.1



AUDIT DATES: March 6th to March 11th, 2026

AUDITED BY: peakbolt

Contents

1 Introduction 2

1.1 About Zenith 3

1.2 Disclaimer 3

1.3 Risk Classification 3

2 Executive Summary 3

2.1 About Meteora 4

2.2 Scope 4

2.3 Audit Timeline 5

2.4 Issues Found 5

3 Findings Summary 5

4 Findings 6

4.1 Informational 7

1

Introduction

1.1 About Zenith

Zenith assembles auditors with proven track records: finding critical vulnerabilities in public audit competitions.

Our audits are carried out by a curated team of the industry's top-performing security researchers, selected for your specific codebase, security needs, and budget.

Learn more about us at https://zenith.security.

1.2 Disclaimer

This report reflects an analysis conducted within a defined scope and time frame, based on provided materials and documentation. It does not encompass all possible vulnerabilities and should not be considered exhaustive.

The review and accompanying report are presented on an "as-is" and "as-available" basis, without any express or implied warranties.

Furthermore, this report neither endorses any specific project or team nor assures the complete security of the project.

1.3 Risk Classification

SEVERITY LEVEL	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

2

Executive Summary

2.1 About Meteora

Our mission is to build the most secure, sustainable and composable liquidity layer for all of Solana and DeFi.

By using Meteora's DLMM and Dynamic AMM Pools, liquidity providers can earn the best fees and yield on their capital.

This would help transform Solana into the ultimate trading hub for mainstream users in crypto by driving sustainable, long-term liquidity to the platform. Join us at Meteora to shape Solana's future as the go-to destination for all crypto participants.

2.2 Scope

The engagement involved a review of the following targets:

Target	dynamic-bonding-curve
Repository	https://github.com/MeteoraAg/dynamic-bonding-curve
Commit Hash	`93fd1a18ad65b9b4c961786d223341024ce9ad9a`
Files	`Changes in PR-174`

Target	Mitigation Review
Repository	https://github.com/MeteoraAg/dynamic-bonding-curve
Commit Hash	`4dacc07f1187f7a9eda06896faa29aaebd104a5e`
Files	`Changes in PR-174`

2.3 Audit Timeline

March 6, 2026	Audit start
March 11, 2026	Audit end
March 12, 2026	Report published

2.4 Issues Found

SEVERITY	COUNT
Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	0
Informational	1
Total Issues	**1**

3

Findings Summary

ID	Description	Status
I-1	Duplicate logic in concentrated liquidity calculate_liquidity_delta and calculate_concentrated_initial_liquidity	Resolved

4

Findings

4.1 Informational

A total of 1 informational findings were identified.

[I-1] Duplicate logic in concentrated liquidity `calculate_liquidity_delta` and `calculate_concentrated_initial_liquidity`

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- concentrated_liquidity.rs#L159-L175
- concentrated_liquidity.rs#L63-L88

Description:

The same logic is duplicated in both `calculate_liquidity_delta()` and `calculate_concentrated_initial_liquidity()` for `ConcentratedLiquidity`.

Consider refactoring away the duplicate code by having `calculate_liquidity_delta()` delegate to `calculate_concentrated_initial_liquidity()` and keep the shared logic in one place.

```
fn calculate_liquidity_delta(
    &self,
    base_amount: u64,
    quote_amount: u64,
    _pool_base_reserve: u64,
    _pool_quote_reserve: u64,
    _pool_liquidity: u128,
) → Result<u128> {
    let liquidity_from_base = get_initial_liquidity_from_delta_base(
        base_amount,
        MAX_SQRT_PRICE,
        self.migration_sqrt_price,
    )?;
    let liquidity_from_quote = get_initial_liquidity_from_delta_quote(
        quote_amount,
```

```
        MIN_SQRT_PRICE,
        self.migration_sqrt_price,
    )?;
    if liquidity_from_base > U512::from(liquidity_from_quote) {
        Ok(liquidity_from_quote)
    } else {
        Ok(liquidity_from_base
            .try_into()
            .map_err(|_| PoolError::TypeCastFailed)?)
    }
}
```

```
pub fn calculate_concentrated_initial_liquidity(
    base_amount: u64,
    quote_amount: u64,
    migration_sqrt_price: u128,
) -> Result<u128> {
    let liquidity_from_base =
        get_initial_liquidity_from_delta_base(base_amount, MAX_SQRT_PRICE,
    migration_sqrt_price)?;
    let liquidity_from_quote =
        get_initial_liquidity_from_delta_quote(quote_amount, MIN_SQRT_PRICE,
    migration_sqrt_price)?;
    if liquidity_from_base > U512::from(liquidity_from_quote) {
        Ok(liquidity_from_quote)
    } else {
        Ok(liquidity_from_base
            .try_into()
            .map_err(|_| PoolError::TypeCastFailed)?)
    }
}
```

Meteora: Resolved with `@4dacc07f11 ...`

Zenith: Verified.